

# Deep Learning School

**Физтех-Школа Прикладной математики и информатики (ФПМИ) МФТИ**

---

*Some parts of the notebook are almost the copy of [mmta-team course](). Special thanks to mmta-team for making them publicly available. [Original notebook]().*

**Прочитайте семинар, пожалуйста, для успешного выполнения домашнего задания. В конце ноутка напишите свой вывод. Работа без вывода оценивается ниже.**

## Задача поиска схожих по смыслу предложений

Мы будем ранжировать вопросы StackOverflow на основе семантического векторного представления

До этого в курсе не было речи про задачу ранжировния, поэтому введем математическую формулировку

## Задача ранжирования(Learning to Rank)

- $X$ - множество объектов
- $X^l = \{x_1, x_2, \dots, x_l\}$ - обучающая выборка

На обучающей выборке задан порядок между некоторыми элементами, то есть нам известно, что некий объект выборки более релевантный для нас, чем другой:

- $i \prec j$ - порядок пары индексов объектов на выборке $X^l$ с индексами $i$ и $j$

### Задача:

построить ранжирующую функцию $a : X \to R$ такую, что

$$i \prec j \Rightarrow a(x_i) < a(x_j)$$

## Embeddings

Будем использовать предобученные векторные представления слов на постах Stack Overflow.
A word2vec model trained on Stack Overflow posts

```
!wget -q https://zenodo.org/record/1199620/files/SO_vectors_200.bin?download=1
```

```
!pip uninstall -y -q gensim
!pip install -q gensim
```

```
WARNING: Skipping gensim as it is not installed.
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 51.9 MB/s eta 0:00:00
```

```
# !pip cache purge
```

```
import gensim
from gensim.models import Word2Vec

print(f"Gensim version: {gensim.__version__}")

# Проверим импорт smart_open
import smart_open
print(f"Smart open version: {smart_open.__version__}")

# Если все ок, продолжаем работу
print("Библиотеки загружены успешно!")
```

```
Gensim version: 4.4.0
Smart open version: 7.5.1
Библиотеки загружены успешно!
```

```
from gensim.models.keyedvectors import KeyedVectors
wv_embeddings = KeyedVectors.load_word2vec_format("SO_vectors_200.bin?download=1", binary=True)
```

## Как пользоваться этими векторами?

Посмотрим на примере одного слова, что из себя представляет embedding

```
word = 'dog'
if word in wv_embeddings:
    print(wv_embeddings[word].dtype, wv_embeddings[word].shape)
```

```
float32 (200,)
```

```
print(f"Num of words: {len(wv_embeddings.index_to_key)}")
```

```
Num of words: 1787145
```

Найдем наиболее близкие слова к слову `dog`:

### *Вопрос 1:*

- Входит ли слово `cat` в топ-5 близких слов к слову `dog`? Какое место оно занимает?

```
# method most_simmilar
similar_words = wv_embeddings.most_similar('dog', topn=5)
for i, (word, similarity) in enumerate(similar_words, 1):
    print(f"{i}. {word}: {similarity:.4f}")
```

```
1. animal: 0.8564
2. dogs: 0.7881
3. mammal: 0.7624
4. cats: 0.7621
5. animals: 0.7608
```

***Ваш ответ:*** 'cat' входит в топ-5 ближайших слов к 'dog' и занимает в топе 4-е место.

## Векторные представления текста

Перейдем от векторных представлений отдельных слов к векторным представлениям вопросов, как к **среднему** векторов всех слов в вопросе. Если для какого-то слова нет предобученного вектора, то его нужно пропустить. Если вопрос не содержит ни одного известного слова, то нужно вернуть нулевой вектор.

```
!pip install -q spacy
```

```
!python -m spacy download en_core_web_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 80.1 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
import spacy
import numpy as np

nlp = spacy.load('en_core_web_sm')
# import numpy as np
# import re
# # you can use your tokenizer
# # for example, from nltk.tokenize import WordPunctTokenizer
# class MyTokenizer:
#     def __init__(self):
#         pass
#     def tokenize(self, text):
#         return re.findall('\w+', text)
# tokenizer = MyTokenizer()
```

```
def question_to_vec(question, embeddings, tokenizer, dim=200):
    """
        question: строка
        embeddings: наше векторное представление
        dim: размер любого вектора в нашем представлении

        return: векторное представление для вопроса
    """
    # вопрос приводим к нижнему регистру, токенизируем
    doc = nlp(question.lower())
    # щас соберем векторы для слов
    word_vecs = []

    # пропускаем знаки препинания и стоп-слова
    for token in doc:
      if not token.is_punct and not token.is_stop:
        word = token.text
        if word in embeddings:
          word_vecs.append(embeddings[word])

    # если нет слова которое мы бы знали то возвращаем нулевой вектор
    if len(word_vecs) == 0:
      return np.zeros(embeddings.vector_size)

    return np.mean(word_vecs, axis=0)
```

Теперь у нас есть метод для создания векторного представления любого предложения.

### *Вопрос 2:*

- Какая третья (с индексом 2) компонента вектора предложения `I love neural networks` (округлите до 2 знаков после запятой)?

```
# Предложение
question = "I love neural networks"

question_vector = question_to_vec(question, wv_embeddings, nlp)
print(round(question_vector[2], 2))
```

```
-1.29
```

## Оценка близости текстов

Представим, что мы используем идеальные векторные представления слов. Тогда косинусное расстояние между дублирующими предложениями должно быть меньше, чем между случайно взятыми предложениями.

Сгенерируем для каждого из $N$ вопросов $R$ случайных отрицательных примеров и примешаем к ним также настоящие дубликаты. Для каждого вопроса будем ранжировать с помощью нашей модели $R + 1$ примеров и смотреть на позицию дубликата. Мы хотим, чтобы дубликат был первым в ранжированном списке.

Hits@K

Первой простой метрикой будет количество корректных попаданий для какого-то $K$:

$$\text{Hits@K} = \frac{1}{N}\sum_{i=1}^{N}[rank_q_i^{'} \leq K],$$

- $[x < 0] \equiv \begin{cases} 1, & x < 0 \\ 0, & x \geq 0 \end{cases}$ - индикаторная функция
- $q_i$ - $i$-ый вопрос
- $q_i^{'}$ - его дубликат
- $rank_q_i^{'}$ - позиция дубликата в ранжированном списке ближайших предложений для вопроса $q_i$.

Hits@K измеряет долю вопросов, для которых правильный ответ попал в топ-K позиций среди отранжированных кандидатов.

DCG@K

Второй метрикой будет упрощенная DCG метрика, учитывающая порядок элементов в списке путем домножения релевантности элемента на вес равный обратному логарифму номера позиции::

$$\text{DCG@K} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\log_2(1+rank_q_i^{'})}\cdot[rank_q_i^{'} \leq K],$$

С такой метрикой модель штрафуется за большой ранк корректного ответа.

DCG@K измеряет качество ранжирования, учитывая не только факт наличия правильного ответа в топ-K, но и ***его точную позицию***.

## Пример оценок

Вычислим описанные выше метрики для игрушечного примера. Пусть

- $N = 1, R = 3$
- "Что такое python?" - вопрос $q_1$
- "Что такое язык python?" - его дубликат $q_i^{'}$

Пусть модель выдала следующий ранжированный список кандидатов:

1. "Как изучить c++?"
2. "Что такое язык python?"
3. "Хочу учить Java"
4. "Не понимаю Tensorflow"

$\Rightarrow rank_q_i^{'} = 2$

Вычислим метрику *Hits@K* для *K = 1, 4*:

- [K = 1] $\text{Hits@1} = [rank_q_i^{'} \leq 1]$

Проверяем условие $\text{rank}_{q_1^{'}} \leq 1$: ***условие неверно***.

Следовательно, $[\text{rank}_{q_1^{'}} \leq 1] = 0$.

- [K = 4] $\text{Hits@4} = [rank_q_i^{'} \leq 4] = 1$

Проверяем условие $\text{rank}_{q_1^{'}} \leq 4$: ***условие верно***.

Вычислим метрику *DCG@K* для *K = 1, 4*:

- [K = 1] $\text{DCG@1} = \frac{1}{\log_2(1+2)}\cdot[2 \leq 1] = 0$
- [K = 4] $\text{DCG@4} = \frac{1}{\log_2(1+2)}\cdot[2 \leq 4] = \frac{1}{\log_2 3}$

## ***Вопрос 3***:

- Вычислите `DCG@10`, если $rank_q_i^{'} = 9$(округлите до одного знака после запятой)

[rank_q'i<= 10] вернет 1, дальше

[K = 10] $\text{DCG@10} = \frac{1}{\log_2(1+9)}\cdot[9 \leq 10] = \frac{1}{\log_2 10}$

ответ: 0.3

## Более сложный пример оценок

Рассмотрим пример с $N > 1$, где $N = 3$ (три вопроса) и для каждого вопроса заданы позиции их дубликатов. Вычислим метрики **Hits@K** для разных значений $K$.

- $N = 3$: Три вопроса ($q_1, q_2, q_3$).
- Для каждого вопроса известна позиция его дубликата ($\text{rank}_{q_i'}$):
  - $\text{rank}_{q_1'} = 2$,
  - $\text{rank}_{q_2'} = 5$,
  - $\text{rank}_{q_3'} = 1$.

Мы будем вычислять **Hits@K** для $K = 1, 5$.

---

**Для $K = 1$:**

Подставим значения:

$$\text{Hits@1} = \frac{1}{3} \cdot \left([\text{rank}_{q_1'} \leq 1] + [\text{rank}_{q_2'} \leq 1] + [\text{rank}_{q_3'} \leq 1]\right).$$

Проверяем условие $\text{rank}_{q_i'} \leq 1$ для каждого вопроса:

- $\text{rank}_{q_1'} = 2 \rightarrow 2 \not\leq 1 \rightarrow 0$,
- $\text{rank}_{q_2'} = 5 \rightarrow 5 \not\leq 1 \rightarrow 0$,
- $\text{rank}_{q_3'} = 1 \rightarrow 1 \leq 1 \rightarrow 1$.

Сумма:

$$\text{Hits@1} = \frac{1}{3} \cdot (0 + 0 + 1) = \frac{1}{3}.$$

$$\boxed{\text{Hits@1} = \frac{1}{3}}.$$

---

**Для $K = 5$:**

Подставим значения:

$$\text{Hits@5} = \frac{1}{3} \cdot \left([\text{rank}_{q_1'} \leq 5] + [\text{rank}_{q_2'} \leq 5] + [\text{rank}_{q_3'} \leq 5]\right).$$

Проверяем условие $\text{rank}_{q_i'} \leq 5$ для каждого вопроса:

- $\text{rank}_{q_1'} = 2 \rightarrow 2 \leq 5 \rightarrow 1$,
- $\text{rank}_{q_2'} = 5 \rightarrow 5 \leq 5 \rightarrow 1$,
- $\text{rank}_{q_3'} = 1 \rightarrow 1 \leq 5 \rightarrow 1$.

Сумма:

$$\text{Hits@5} = \frac{1}{3} \cdot (1 + 1 + 1) = 1.$$

$$\boxed{\text{Hits@5} = 1}.$$

Теперь вычислим метрику **DCG@K** для того же примера, где $N = 3$ (три вопроса), и для каждого вопроса известна позиция его дубликата ($\text{rank}_{q_i'}$):

- $\text{rank}_{q_1'} = 2$,
- $\text{rank}_{q_2'} = 5$,
- $\text{rank}_{q_3'} = 1$.

Мы будем вычислять **DCG@K** для $K = 1, 5$.

---

**Для $K = 1$:** Подставим значения:

$$\text{DCG@1} = \frac{1}{3} \cdot \left( \frac{1}{\log_2(1 + \text{rank}_{q_1'})} \cdot [\text{rank}_{q_1'} \leq 1] + \frac{1}{\log_2(1 + \text{rank}_{q_2'})} \cdot [\text{rank}_{q_2'} \leq 1] + \frac{1}{\log_2(1 + \text{rank}_{q_3'})} \cdot [\text{rank}_{q_3'} \leq 1] \right).$$

Проверяем условие $\text{rank}_{q_i'} \leq 1$ для каждого вопроса:

- $\text{rank}_{q_1'} = 2 \rightarrow 2 \not\leq 1 \rightarrow 0$,
- $\text{rank}_{q_2'} = 5 \rightarrow 5 \not\leq 1 \rightarrow 0$,
- $\text{rank}_{q_3'} = 1 \rightarrow 1 \leq 1 \rightarrow 1$.

Сумма:

$$\text{DCG@1} = \frac{1}{3} \cdot (0 + 0 + 1) = \frac{1}{3}.$$

$$\boxed{\text{DCG@1} = \frac{1}{3}}.$$

---

**Для $K = 5$:** Подставим значения:

$$\text{DCG@5} = \frac{1}{3} \cdot \left( \frac{1}{\log_2(1 + \text{rank}_{q_1'})} \cdot [\text{rank}_{q_1'} \leq 5] + \frac{1}{\log_2(1 + \text{rank}_{q_2'})} \cdot [\text{rank}_{q_2'} \leq 5] + \frac{1}{\log_2(1 + \text{rank}_{q_3'})} \cdot [\text{rank}_{q_3'} \leq 5] \right).$$

Проверяем условие $\text{rank}_{q_i'} \leq 5$ для каждого вопроса:

- $\text{rank}_{q_1'} = 2 \rightarrow 2 \leq 5 \rightarrow 1$,
- $\text{rank}_{q_2'} = 5 \rightarrow 5 \leq 5 \rightarrow 1$,
- $\text{rank}_{q_3'} = 1 \rightarrow 1 \leq 5 \rightarrow 1$.

Сумма:

$$\text{DCG@5} = \frac{1}{3} \cdot (0.631 + 0.387 + 1) = \frac{1}{3} \cdot 2.018 \approx 0.673.$$

$$\boxed{\text{DCG@5} \approx 0.673}.$$

***Вопрос 4:***

- Найдите максимум `Hits@47 - DCG@1`?

## HITS_COUNT и DCG_SCORE

Каждая функция имеет два аргумента: *dup_ranks* и *k*.

*dup_ranks* является списком, который содержит рейтинги дубликатов (их позиции в ранжированном списке).

К примеру для "Что такое язык python?" $dup_ranks = [2]$.

```
def hits_count(dup_ranks, k):
    """
        dup_ranks: list индексов дубликатов
        k: пороговое значение для ранга
        result: вернуть Hits@k
    """
    # Подсчитываем количество дубликатов, чей ранг <= k
    count = sum(1 for rank in dup_ranks if rank <= k)
    hits_value = count / len(dup_ranks) if dup_ranks else 0
    return hits_value
```

```
dup_ranks = [2]

k = 1
hits_value = hits_count(dup_ranks, k)
print(f"Hits@1 = {hits_value}")

k = 4
hits_value = hits_count(dup_ranks, k)
print(f"Hits@4 = {hits_value}")
```

```
Hits@1 = 0.0
Hits@4 = 1.0
```

```
import math

def dcg_score(dup_ranks, k):
    """
        dup_ranks: list индексов дубликатов
        k: пороговое значение для ранга
        result: вернуть DCG@k
    """
    if not dup_ranks:
      return 0.0

    dcg_sum = 0
    # Вычисляем сумму для всех релевантных дубликатов
    for rank in dup_ranks:
      if rank <= k:
        dcg_sum += 1/math.log2(1 + rank)

    # Делим на общее количество вопросов
    dcg_value = dcg_sum / len(dup_ranks)
    return dcg_value
```

```
# Пример списка позиций дубликатов
dup_ranks = [2]

# Вычисляем DCG@1
dcg_value = dcg_score(dup_ranks, k=1)
print(f"DCG@1 = {dcg_value:.3f}")
```

```
# Вычисляем DCG@4
dcg_value = dcg_score(dup_ranks, k=4)
print(f"DCG@10 = {dcg_value:.3f}")
```

```
DCG@1 = 0.000
DCG@10 = 0.631
```

Протестируем функции. Пусть $N = 1$, то есть один эксперимент. Будем искать копию вопроса и оценивать метрики.

```
import pandas as pd
```

```
copy_answers = ["How does the catch keyword determine the type of exception that was thrown",]

# наши кандидаты
candidates_ranking = [["How Can I Make These Links Rotate in PHP",
                       "How does the catch keyword determine the type of exception that was thrown",
                       "NSLog array description not memory address",
                       "PECL_HTTP not recognised php ubuntu"],]

# dup_ranks — позиции наших копий, так как эксперимент один, то этот массив длины 1
dup_ranks = [2]

# вычисляем метрику для разных k
print('Ваш ответ HIT:', [hits_count(dup_ranks, k) for k in range(1, 5)])
print('Ваш ответ DCG:', [round(dcg_score(dup_ranks, k), 5) for k in range(1, 5)])
```

```
Ваш ответ HIT: [0.0, 1.0, 1.0, 1.0]
Ваш ответ DCG: [0.0, 0.63093, 0.63093, 0.63093]
```

У вас должно получиться

```
# correct_answers - метрика для разных k
correct_answers = pd.DataFrame([[0, 1, 1, 1], [0, 1 / (np.log2(3)), 1 / (np.log2(3)), 1 / (np.log2(3))]],
                               index=['HITS', 'DCG'], columns=range(1,5))
correct_answers
```

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| **HITS** | 0 | 1.00000 | 1.00000 | 1.00000 |
| **DCG** | 0 | 0.63093 | 0.63093 | 0.63093 |

## Данные

arxiv link

`train.tsv` - выборка для обучения.
В каждой строке через табуляцию записаны: **<вопрос>, <похожий вопрос>**

`validation.tsv` - тестовая выборка.
В каждой строке через табуляцию записаны: **<вопрос>, <похожий вопрос>, <отрицательный пример 1>, <отрицательный пример 2>, ...**

```
!unzip stackoverflow_similar_questions.zip
```

```
Archive:  stackoverflow_similar_questions.zip
   creating: data/
  inflating: data/.DS_Store
   creating: __MACOSX/
   creating: __MACOSX/data/
  inflating: __MACOSX/data/._.DS_Store
  inflating: data/train.tsv
  inflating: data/validation.tsv
```

Считайте данные.

```
def read_corpus(filename):
    data = []
    with open(filename, encoding='utf-8') as file:
        for line in file:
            data.append(line.strip().split('\t'))
    return data
```

Нам понадобиться только файл validation.

```
validation_data = read_corpus('./data/validation.tsv')
```

Кол-во строк

```
len(validation_data)
```

```
3760
```

Размер нескольких первых строк

```
for i in range(25):
    print(i + 1, len(validation_data[i]))
```

```
1 1001
2 1001
3 1001
4 1001
5 1001
6 1001
7 1001
8 1001
9 1001
10 1001
11 1001
12 1001
13 1001
14 1001
15 1001
16 1001
17 1001
18 1001
19 1001
20 1001
21 1001
22 1001
23 1001
24 1001
25 1001
```

## Ранжирование без обучения

Реализуйте функцию ранжирования кандидатов на основе косинусного расстояния. Функция должна по списку кандидатов вернуть отсортированный список пар (позиция в исходном списке кандидатов, кандидат). При этом позиция кандидата в полученном списке является его рейтингом (первый - лучший). Например, если исходный список кандидатов был [a, b, c], и самый похожий на исходный вопрос среди них - c, затем a, и в конце b, то функция должна вернуть список **[(2, c), (0, a), (1, b)]**.

```
from sklearn.metrics.pairwise import cosine_similarity
from copy import deepcopy
```

```
def rank_candidates(question, candidates, embeddings, tokenizer, dim=200):
    """
        question: строка
        candidates: массив строк (кандидатов) [a, b, c]
        result: пары (начальная позиция, кандидат) [(2, c), (0, a), (1, b)]
    """
    # целевой вопрос
    question_vec = question_to_vec(question, embeddings, tokenizer)
    candidate_vecs = []
    # эмбеддинги всех вопросов кандидатов
    for cand in candidates:
      cand_vec = question_to_vec(cand, embeddings, tokenizer)
      candidate_vecs.append(cand_vec)

    # преобразуем в 2д массивы для косинусного расстояния
    question_vec_2d = question_vec.reshape(1, -1)
    candidate_vecs_2d  = np.array(candidate_vecs)

    # id, сходство, кандидат
    similarities = cosine_similarity(question_vec_2d, candidate_vecs_2d)[0]
    cand_with_score = [(i, similarities[i], candidates[i]) for i in range(len(candidates))]

    cand_with_score.sort(key=lambda x: x[1], reverse=True)

    result = [(pos, candidate) for pos, sim, candidate in cand_with_score]

    return result
```

Протестируйте работу функции на примерах ниже. Пусть $N = 2$, то есть два эксперимента

```
questions = ['converting string to list', 'Sending array via Ajax fails']

candidates = [['Convert Google results object (pure js) to Python object', # первый эксперимент
               'C# create cookie from string and send it',
               'How to use jQuery AJAX for an outside domain?'],

              ['Getting all list items of an unordered list in PHP',      # второй эксперимент
               'WPF- How to update the changes in list item of a list',
               'select2 not displaying search results']]
```

```
for question, q_candidates in zip(questions, candidates):
        ranks = rank_candidates(question, q_candidates, wv_embeddings, nlp)
        print(ranks)
        print()
```

```
[(1, 'C# create cookie from string and send it'), (0, 'Convert Google results object (pure js) to Python object'), (2, 'How

[(0, 'Getting all list items of an unordered list in PHP'), (2, 'select2 not displaying search results'), (1, 'WPF- How to u
```

Для первого экперимента вы можете полностью сравнить ваши ответы и правильные ответы. Но для второго эксперимента два ответа на кандидаты будут **скрыты**(*)

```
# # должно вывести
# results = [[(1, 'C# create cookie from string and send it'),
#             (0, 'Convert Google results object (pure js) to Python object'),
#             (2, 'How to use jQuery AJAX for an outside domain?')],
#            [(*, 'Getting all list items of an unordered list in PHP'), #скрыт
#             (*, 'select2 not displaying search results'), #скрыт
#             (*, 'WPF- How to update the changes in list item of a list')]] #скрыт
```

Последовательность начальных индексов вы должны получить `для эксперимента 1` 1, 0, 2.

### *Вопрос 5:*

- Какую последовательность начальных индексов вы получили `для эксперимента 2`(перечисление без запятой и пробелов, например, `102` для первого эксперимента?

```
# 021
```

```
  File "/tmp/ipykernel_1570/2205270298.py", line 1
    021
    ^
SyntaxError: leading zeros in decimal integer literals are not permitted; use an 0o prefix for octal integers
```

Теперь мы можем оценить качество нашего метода. Запустите следующие два блока кода для получения результата. Обратите внимание, что вычисление расстояния между векторами занимает некоторое время (примерно 10 минут). Можете взять для validation 1000 примеров.

```
from tqdm.notebook import tqdm
```

```
wv_ranking = []
max_validation_examples = 100
for i, line in enumerate(tqdm(validation_data)):
    if i == max_validation_examples:
        break
    q, *ex = line
    ranks = rank_candidates(q, ex, wv_embeddings, nlp)
    wv_ranking.append([r[0] for r in ranks].index(0) + 1)
```

```
  3%                                                 100/3760 [11:41<6:55:26,  6.81s/it]
```

```
for k in tqdm([1, 5, 10, 100, 500, 1000]):
    print("DCG@%4d: %.3f | Hits@%4d: %.3f" % (k, dcg_score(wv_ranking, k), k, hits_count(wv_ranking, k)))
```

```
100%                                                  6/6 [00:00<00:00, 410.60it/s]
DCG@   1: 0.410 | Hits@   1: 0.410
DCG@   5: 0.489 | Hits@   5: 0.570
DCG@  10: 0.514 | Hits@  10: 0.650
DCG@ 100: 0.568 | Hits@ 100: 0.910
DCG@ 500: 0.578 | Hits@ 500: 0.990
DCG@1000: 0.579 | Hits@1000: 1.000
```

Из формул выше можно понять, что

- $\mathrm{Hits@K}$ **монотонно неубывающая функция** $K$, которая стремится к 1 при $K \to \infty$.
- $\mathrm{DCG@K}$ **монотонно неубывающая функция** $K$, но рост замедляется с увеличением $K$ из-за убывания веса $\frac{1}{\log_2(1+\mathrm{rank}_{q_i'})}$.

## Эмбеддинги, обученные на корпусе похожих вопросов

```
train_data = read_corpus('./data/train.tsv')
```

Улучшите качество модели.
Склеим вопросы в пары и обучим на них модель Word2Vec из gensim. Выберите размер window. Объясните свой выбор.

***Рассмотрим подробнее*** данное склеивание.

1. Каждая строка из train_data разбивается на вопрос (question) и список кандидатов.
2. Для каждого кандидата вопрос склеивается с ним в одну строку.
3. Склеенная строка (combined_text) токенизируется, и полученный список токенов добавляется в общий корпус (corpus).

***Пример***

```
Вопрос: "What is Python?"
Кандидаты: ["Python is a programming language", "Java is another language"]
Склеенные строки:
    "What is Python? Python is a programming language"
    "What is Python? Java is another language"

Токенизированные списки:
    ['what', 'is', 'python', 'python', 'is', 'a', 'programming', 'language']
    ['what', 'is', 'python', 'java', 'is', 'another', 'language']
```

```
train_data[111254]
```

```
['Generators - Printing generated values',
 'Understanding Generators in Python']
```

```
# создаем корпус текстов
corpus = []
error_count = 0

for i, line in enumerate(tqdm(train_data, desc="Создание корпуса")):
    try:
        question = line[0]
        similar = line[1]

        combined_text = question + " " + similar

        doc = nlp(combined_text.lower())

        tokens = [token.text for token in doc if not token.is_punct and not token.is_stop]

        if tokens:
            corpus.append(tokens)

    except IndexError:
        error_count += 1
        if error_count <= 5:
            print(f"Проблемная строка {i}: {line}")
        continue

print(f"Размер корпуса: {len(corpus)} предложений")
print(f"Количество ошибок: {error_count}")
print(f"Пример первых токенов: {corpus[0][:20] if corpus else 'Нет данных'}")
```

```
Создание корпуса: 100%                                                  1000000/1000000 [2:29:02<00:00, 130.82it/s]
Размер корпуса: 1000000 предложений
Количество ошибок: 0
Пример первых токенов: ['converting', 'string', 'list', 'convert', 'google', 'results', 'object', 'pure', 'js', 'python', 'c
```

```
from gensim.models import Word2Vec
embeddings_trained = Word2Vec(
    sentences=corpus,        # Корпус токенизированных текстов
    vector_size=200,         # Размерность векторов
    window=5,                # Размер окна контекста
    min_count=5,             # Минимальная частота слов
    workers=4,               # Количество потоков
    epochs=10,               # количество эпох
    sg=0
).wv
```

```
wv_ranking = []
max_validation_examples = 100
for i, line in enumerate(tqdm(validation_data)):
    if i == max_validation_examples:
        break
    q, *ex = line
    ranks = rank_candidates(q, ex, embeddings_trained, nlp)
    wv_ranking.append([r[0] for r in ranks].index(0) + 1)
```

```
  3%                                                    100/3760 [11:39<7:07:02,  7.00s/it]
```

```
for k in tqdm([1, 5, 10, 100, 500, 1000]):
    print("DCG@%4d: %.3f | Hits@%4d: %.3f" % (k, dcg_score(wv_ranking, k), k, hits_count(wv_ranking, k)))
```

```
100%                                                    6/6 [00:00<00:00, 468.52it/s]
DCG@   1: 0.410 | Hits@   1: 0.410
DCG@   5: 0.528 | Hits@   5: 0.630
DCG@  10: 0.545 | Hits@  10: 0.680
DCG@ 100: 0.594 | Hits@ 100: 0.910
DCG@ 500: 0.606 | Hits@ 500: 1.000
DCG@1000: 0.606 | Hits@1000: 1.000
```

Замечание:

Решить эту задачу с помощью обучения полноценной нейронной сети будет вам предложено, как часть задания в одной из домашних работ по теме "Диалоговые системы".

Напишите свой вывод о полученных результатах.

- Какой принцип токенизации даёт качество лучше и почему?
- Помогает ли нормализация слов?
- Какие эмбеддинги лучше справляются с задачей и почему?
- Почему получилось плохое качество решения задачи?
- Предложите свой подход к решению задачи.

Вывод: токенизация с удалением стоп-слов и знаков препинания, использованием лемматизации или стемминга и приведением слов к нижнему регистру. Да, помогает, уменьшает размер словаря и улучшает обобщение. Моя модель работает лучше предобученной, потому что модель обучалась на парах вопросов, которые похожи друг на друга. Размер корпуса для дообучения был недостаточным, нет учета важности таких слов, как how, what, которые дают информационный импакт вопросу.

Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")